DDL (Data Definition Language) Commands in Oracle

DDL commands are used to **define, modify, and manage database objects** such as tables, indexes, and schemas. The key DDL commands in Oracle are:

---

1. CREATE – Used to create database objects

Example: Creating a Table

```
CREATE TABLE EMPLOYEES (
    EMP_ID NUMBER PRIMARY KEY,
    EMP_NAME VARCHAR2(50),
    HIRE_DATE DATE,
    SALARY NUMBER(10,2)
);
```

✅ **Creates a table named `EMPLOYEES` with columns.**

---

2. ALTER – Used to modify an existing object

Example: Adding a Column

```
ALTER TABLE EMPLOYEES ADD (DEPARTMENT_ID NUMBER);
```

✅ **Adds a new column `DEPARTMENT_ID` to the `EMPLOYEES` table.**

Example: Modifying a Column

```
ALTER TABLE EMPLOYEES MODIFY (SALARY NUMBER(12,2));
```

✅ **Changes the data type and size of the `SALARY` column.**

Example: Dropping a Column

```
ALTER TABLE EMPLOYEES DROP COLUMN DEPARTMENT_ID;
```

✅ **Removes `DEPARTMENT_ID` from the table.**

---

3. DROP – Used to delete database objects permanently

Example: Dropping a Table

```
DROP TABLE EMPLOYEES;
```

✅ **Deletes the `EMPLOYEES` table and all its data permanently.**

---

4. TRUNCATE – Used to delete all rows from a table

Example: Truncating a Table

```
TRUNCATE TABLE EMPLOYEES;
```

✅ **Deletes all rows from `EMPLOYEES`, but keeps the structure intact.**
⚠ Unlike `DELETE`, you **cannot rollback** a `TRUNCATE`.

---

5. RENAME – Used to rename a database object

Example: Renaming a Table

```
RENAME EMPLOYEES TO STAFF;
```

✅ **Changes the table name from `EMPLOYEES` to `STAFF`.**

---

6. COMMENT – Used to add comments to database objects

Example: Adding a Comment to a Table

```
COMMENT ON TABLE EMPLOYEES IS 'Stores employee details';
```

✅ **Adds a comment to the `EMPLOYEES` table.**

Example: Adding a Comment to a Column

```
COMMENT ON COLUMN EMPLOYEES.SALARY IS 'Stores employee salary';
```

✅ **Adds a comment to the `SALARY` column in `EMPLOYEES`.**

Key Differences Between DDL and DML

Feature	DDL (Data Definition Language)	DML (Data Manipulation Language)
Purpose	Defines structure of database objects	Manipulates data inside tables
Commands	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`, `COMMENT`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
Rollback	✗ No rollback (auto-commit)	☑ Can rollback (except `TRUNCATE`)

DML (Data Manipulation Language) Commands in Oracle SQL

DML commands in SQL are used to manipulate and manage data in database tables. The main DML commands in Oracle SQL are:

1. **INSERT** – Adds new records to a table
2. **UPDATE** – Modifies existing records in a table
3. **DELETE** – Removes records from a table
4. **MERGE** – Inserts, updates, or deletes records based on conditions

---

1. INSERT Command

The `INSERT` statement is used to add new rows to a table.

Syntax:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Example:

```
INSERT INTO employees (employee_id, first_name, last_name, salary,
department_id) VALUES (101, 'John', 'Doe', 50000, 10);
```

Inserting Multiple Rows (Oracle 11g and later)

```
INSERT ALL
    INTO employees (employee_id, first_name, last_name, salary,
department_id) VALUES (102, 'Jane', 'Smith', 55000, 20)
    INTO employees (employee_id, first_name, last_name, salary,
department_id) VALUES (103, 'Mike', 'Brown', 60000, 30) SELECT * FROM dual;
```

How to insert values into table by using & symbol in Oracle at runtime:

You can use the `&` symbol in Oracle SQL to take input values at runtime when inserting records into a table. Here's how you can do it:

General Syntax

Optional

```
INSERT INTO table_name (column1, column2, column3)
VALUES (&value1, '&value2', TO_DATE('&value3', 'YYYY-MM-DD'));
```

Example: Inserting Employee Data

Assume you have a table named `EMPLOYEES` with columns `EMP_ID` (NUMBER), `EMP_NAME` (VARCHAR2), and `HIRE_DATE` (DATE).

```
INSERT INTO EMPLOYEES (EMP_ID, EMP_NAME, HIRE_DATE)
VALUES (&emp_id, '&emp_name', TO_DATE('&hire_date', 'YYYY-MM-DD'));
```

How It Works

1. When you run the query, Oracle will prompt you to enter values for:
   - `&emp_id` → Example: `101`
   - `&emp_name` → Example: `John Doe`
   - `&hire_date` → Example: `2024-02-01`
2. The `TO_DATE('&hire_date', 'YYYY-MM-DD')` ensures the date is correctly formatted before insertion.
3. The record gets inserted into the table.

Last executed statement in the Buffer

For inserting multiple rows at runtime, we can use / symbol at SQL prompt after insertion of first record, the prompt will repeatedly ask data for insertion

In Oracle SQL, you can use the `&` symbol to accept user input at runtime, and to insert a date into a table, you should use the `TO_DATE` function to ensure the correct format. Here's how you can do it:

Example

Assume you have a table named `EMPLOYEES` with a column `HIRE_DATE` of type `DATE`:

```
INSERT INTO EMPLOYEES (EMP_ID, EMP_NAME, HIRE_DATE)
VALUES (&emp_id, '&emp_name', TO_DATE('&hire_date', 'YYYY-MM-DD'));
```

Steps

1. When executing the query, Oracle will prompt you to enter values for `&emp_id`, `&emp_name`, and `&hire_date`.
2. Enter the date in the format `YYYY-MM-DD` (or another expected format).
3. Oracle will convert the input string into a valid `DATE` format using `TO_DATE`.

Alternative Date Formats

If your date format is different, adjust the `TO_DATE` function accordingly:

- `TO_DATE('&hire_date', 'DD-MON-YYYY')` → If you enter `01-JAN-2025`
- `TO_DATE('&hire_date', 'MM/DD/YYYY')` → If you enter `02/01/2025`

2. UPDATE Command

The `UPDATE` statement modifies existing records in a table.

Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Example:

```
UPDATE employees
SET salary = salary + 5000
WHERE department_id = 10;
```

3. DELETE Command

The `DELETE` statement removes one or more records from a table.

Syntax:

```
DELETE FROM table_name
WHERE condition;
```

Example:

```
DELETE FROM employees
WHERE employee_id = 101;
```

Deleting All Rows from a Table (*With WHERE Condition*)

```
DELETE FROM employees;
```

⚠ ***Note: If `WHERE` is omitted, all rows will be deleted, but the table structure remains.***

TCL (Transaction Control Language)

TCL (Transaction Control Language) commands manage transactions in Oracle Database. They ensure *data consistency* and *integrity* during operations.

TCL Commands:

1. **COMMIT** – Saves all changes made by the transaction.
2. **ROLLBACK** – Reverts all changes since the last COMMIT.
3. **SAVEPOINT** – Creates a temporary save point to roll back to a specific point in a transaction.

---

1. COMMIT Command

The `COMMIT` statement saves all changes made during the current transaction permanently in the database.

Syntax:

```
COMMIT;
```

Example:

```
UPDATE employees SET salary = salary + 5000 WHERE department_id = 10;
COMMIT;  -- Changes are saved permanently
```

✅ Once `COMMIT` is executed, changes **cannot** be undone.

---

2. ROLLBACK Command

The `ROLLBACK` statement undoes all changes made in the current transaction since the last `COMMIT`.

Syntax:

```
ROLLBACK;
```

Example:

```
UPDATE employees SET salary = salary + 5000 WHERE department_id = 10;
```

```
ROLLBACK;  -- Reverts salary changes back to original
```

✅ After `ROLLBACK`, the table data is restored to its previous state.

---

3. SAVEPOINT Command

The `SAVEPOINT` statement allows rolling back part of a transaction instead of the entire transaction.

Syntax:

```
SAVEPOINT savepoint_name;
```

Example:

```
UPDATE employees SET salary = salary + 5000 WHERE employee_id = 101;
SAVEPOINT before_bonus;

UPDATE employees SET salary = salary + 3000 WHERE employee_id = 102;
ROLLBACK TO before_bonus;  -- Undo the second update, but keep the first

COMMIT;  -- Save the first update permanently
```

✅ `ROLLBACK TO savepoint_name;` reverts changes **only up to that savepoint**.
✅ The `COMMIT` statement will erase all savepoints.

Practical Example Using All TCL Commands

```
-- Start a transaction
UPDATE employees SET salary = salary + 5000 WHERE employee_id = 101;
SAVEPOINT sp1;

UPDATE employees SET salary = salary + 3000 WHERE employee_id = 102;
SAVEPOINT sp2;

UPDATE employees SET salary = salary + 2000 WHERE employee_id = 103;

-- Rollback to sp2 (Undo last update but keep previous ones)
ROLLBACK TO sp2;

-- Commit changes up to sp2
COMMIT;
```

What Happens?

- Employee **101** gets a **5000** salary increase. ✅
- Employee **102** gets a **3000** salary increase. ✅
- Employee **103** does **not** get a salary increase (rolled back). ❌
- The `COMMIT` ensures changes up to `sp2` are permanent. 🔒